

4주차 · Simulink PID 와 첫 제어기

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-24

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 본 과목 담당 교수가 직접 강의한 것이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

#	과목	수준	언어	영상	자료·코드
1	제어공학 — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	제어시스템설계 — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	캡스톤디자인 — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	제어공학특론 — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	센서신호처리 및 융합 — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB-Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것. 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`

저장소	주소	받는 자리 (WSL)
강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트	https://github.com/wkyouncnu/Capstone-Design	<code>~/Capstone-Design</code>
ROS 2 예제 패키지 <code>usv_basics</code> — 2주차부터 쓰는 노드 코드	https://github.com/wkyouncnu/usv_basics	<code>~/capstone_ws/src/usv_basics</code>

연제	명령 (VS Code 의 WSL 창 터미널)	하는 일
처음 한 번	<code>cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git</code>	강의자료 전체를 받음 (약 400 MB)
처음 한 번	<code>mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git</code>	예제 패키지를 받음. 이어서 <code>cd ~/capstone_ws && colcon build --symlink-install</code>
매주 수업 전	<code>cd ~/Capstone-Design && git pull</code> · <code>cd ~/capstone_ws/src/usv_basics && git pull</code>	바뀐 파일만 받음. 다시 clone 하지 않음
무엇이 바뀌었는지	<code>git log --oneline -10</code> · <code>git show --stat HEAD</code>	교수가 갱신한 내역을 확인

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엮
- `usv_basics` 를 받은 뒤 새 노드가 생겼으면 `colcon build --symlink-install` 을 한 번 더 돌림
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → [강의자료는-한-번-받고-git-pull-로-갱신한다](#)

- 과목: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- 이번 주차 학습 내용: 제어를 배가 아닌 가장 순한 플랜트에서 원리부터 익히고, 같은 제어를 오프라인 WAM-V 의 속도 루프 → 헤딩 루프 → 내부 루프로 옮긴 뒤, 같은 계인을 VRX 에서 확인함

★ 시작 전 확인

- 강의자료부터 갱신: VS Code WSL 창 터미널에서 `cd ~/Capstone-Design && git pull` — 문서와 코드·모델이 같은 판이 됨 ([강의자료는-한-번-받고-git-pull-로-갱신한다](#), 처음 받는 법은 2주차 2-6)
- MATLAB R2024b + Simulink + ROS Toolbox 설치 확인
- 1~4부(1단계~4단계)는 Gazebo 없이 MATLAB 만으로 됨. VRX 는 5단계에서만 필요함
- VRX 기동은 3주차 2-3 의 경량 모드(`w03_vrx_lite/run_vrx.sh`)를 씀
- Simulink 가 처음이면 [A1 Simulink 기초](#) 를 먼저 할 것 — 창·블록·버스·MATLAB Function 사용법은 그 부록에 있고, 이번 주차는 그것을 이미 안다고 전제함

I 이번 주차의 순서 — 다섯 단계

단계	내용	플랜트	Gazebo
1	PID 기본 원리	질량-스프링-댐퍼 하나 (배 아님)	불필요
2	속도 제어	오프라인 WAM-V 종방향	불필요
3	헤딩 제어	오프라인 WAM-V 요측	불필요
4	속도 + 헤딩 내부 루프	오프라인 WAM-V 3자유도	불필요
5	같은 알고리즘을 VRX 로	Gazebo + ROS 2	필요

- 각 단계는 **제어기 없이 돌려 보는 것(개루프)**으로 시작함. 무엇을 제어해야 하는지 먼저 재고, 그 숫자로 계인을 정함
- 2~4단계와 5단계는 **쌍둥이임**. 제어기 상자와 계인이 같고 플랜트 자리만 바뀜

이 주차를 마치면 할 수 있어야 하는 것

1. 자유물체도에서 **운동방정식** → **전달함수**를 손으로 유도하기
2. 2차계의 $\zeta \cdot \omega_n$ 이 응답의 무엇을 정하는지 식과 수치로 설명하기
3. **P · I · D** 가 각각 무엇을 하고 무엇을 못 하는지 스텝응답 수치로 설명하기
4. PID 를 **Gain · Integrator · Sum** 으로 직접 조립하고 라이브러리 블록과 같음을 확인하기
5. **유사미분**과 **안티와인드업**이 왜 필요한지 설명하고 켄 경우·끈 경우를 수치로 비교하기
6. 개루프 계단 추력에서 **정상상태 속도 · 시상수 · 추력 한계 최고속**을 재고 손계산과 맞추기
7. 속도 루프에 **왜 I**가 필요하고 **D**가 해로운지 운동방정식으로 설명하기
8. 개루프 계단 모멘트에서 **선수각이 멈추지 않는 이유**를 설명하기
9. 요측 운동방정식에서 **P-D 극배치 공식**을 유도하고 계인을 계산하기
10. $\pm 180^\circ$ 이음매에서 **ssa**가 없으면 무슨 일이 생기는지 수치로 보이기
11. Simulink 의 **ROS 2 Publish / Subscribe** 블록으로 Gazebo 와 데이터를 주고받기
12. **오프라인 결과와 VRX 결과를** 대조하고 차이가 나는 항목을 지목하기

준비물

항목	내용
환경	1~3주차 WSL2 + ROS 2 + VRX (5단계에서만)
MATLAB	R2024b + Simulink + ROS Toolbox
배포 파일	w04_simulink/ 폴더 (모델 14개 + 스크립트 7개)
파라미터	w04_setup.m 하나 — 1~5단계의 모든 값이 여기 있음

1부 · 이론

1-1. 개루프와 피드백 — 왜 되먹임이 필요한가

★ 제어가 하는 일은 한 문장

재보고, 틀린 만큼 고침. 재보지 않으면 제어가 아니라 단순한 명령임

개루프 — 재보지 않고 내보낸다



조류·바람·짐의 무게가 달라지면 그만큼 어긋난 채로 끝난다. 얼마나 어긋났는지 알 방법도 없다.

피드백(폐루프) — 재보고, 틀린 만큼 고친다

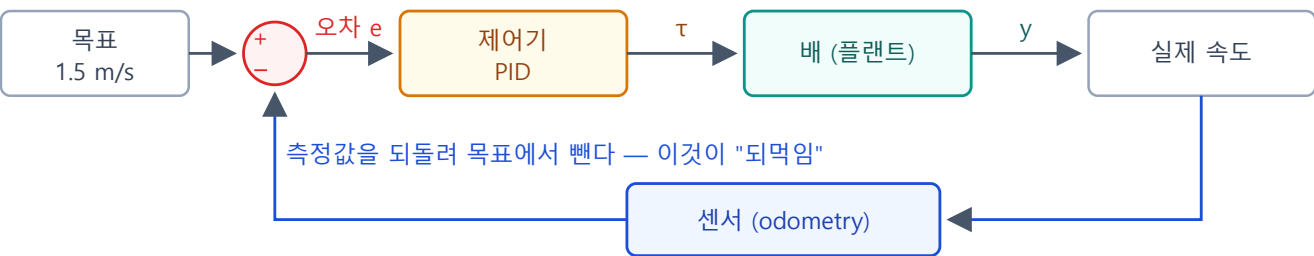


그림 읽는 법

자리	이름	하는 일
맨 왼쪽	목표값 (setpoint)	얼마로 가고 싶은가
동그라미 + -	합산점	목표값에서 측정값을 뺌 → 오차
주황 상자	제어기	오차를 보고 제어입력을 만듦
청록 상자	플랜트	제어입력을 받아 실제로 움직이는 것 (배·모터·수레)
파랑 상자	센서	실제 값을 재서 되돌려 줌

두 방식의 차이

	개루프	피드백(폐루프)
측정	안 함	함
판단 근거	미리 정한 값	지금의 오차
외란(조류·바람·짐)	그만큼 어긋난 채 끝남	어긋난 만큼 스스로 메움
모델이 틀리면	그대로 틀림	어느 정도 덮어 줌
대가	없음	잘못 설계하면 진동하거나 발산함

- 3주차의 "추력 200 N 을 준다"는 개루프였음. 배가 몇 m/s 로 가는지 보지 않았음
- 이번 주차부터는 속도를 재서 목표와 비교함. 그것이 피드백

- 다만 이번 주차의 각 단계는 **개루프로 먼저 돌려 봄**. 제어할 대상이 무엇을 하는지 모르면 계인을 정할 근거가 없기 때문임

! 피드백이 공짜가 아닌 이유

되먹임은 **고칠 힘**을 주지만 동시에 **되돌아오는 길**을 만듦
 너무 세게 고치려 들면 고친 것이 다시 오차가 되어 돌아옴 → 진동
 그래서 "얼마나 세게 고칠 것인가"를 정하는 값이 필요함. 그것이 **게인**

1-2. 제어를 배가 아닌 것에서 먼저 배우는 이유

- 배는 게인 하나를 바꾸면 **여러 가지가 같이 움직임** — 항력, 추진기 포화, 두 루프 사이의 간섭
- 무엇 때문에 달라졌는지 분리되지 않으면 배울 수가 없음
- 그래서 1단계에서는 **출렁이는 것 하나뿐인 가장 순한 2차 시스템**을 씀
- 자전거를 배우려고 오토바이에 올라타지 않는 것과 같음

1단계에서 쓰는 것	값	뜻
플랜트	질량-스프링-댐퍼 (질량 1 kg, 감쇠 2 N-s/m, 스프링 2 N/m)	수레를 용수철로 벽에 매단 것
입력	힘 [N]	제어기가 내는 값
출력	위치 [m]	목표에 맞추려는 값
제어 없이 두면	살짝 출렁이다 멈춤	제어기가 없어도 발산하지 않음

⚠ 1단계의 y 는 동쪽 위치가 아님

3주차부터 y 는 **동쪽 위치**였음. 1단계에서만 y 는 **수레의 위치**, 즉 일반적인 출력임
 배로 돌아가는 1-13절부터는 다시 3주차 규칙(x 북 · y 동)을 씀

1-3. 플랜트를 적는 법 — 그림에서 전달함수까지

현상 — 1 N 으로 밀면 0.5 m 에서 멈춘다

- 수레를 1 N 으로 계속 밀면 끝없이 밀려나지 않고 **0.5 m 에서 멈춤**
- 이유: 밀려난 만큼 스프링이 되밀어, 두 힘이 같아지는 자리에서 멈춤
- 그 자리를 손으로 계산할 수 있어야 제어를 설계할 수 있음

결과 식

$$m\ddot{y} + b\dot{y} + ky = f \quad \Rightarrow \quad G(s) = \frac{Y(s)}{F(s)} = \frac{1}{ms^2 + bs + k}$$

- $m = 1, b = 2, k = 2$ 를 넣으면 $G(s) = \frac{1}{s^2 + 2s + 2}$

유도 — 한 줄씩

1. 수레에 걸리는 힘은 셋임 — 미는 힘 f , 스프링이 되미는 힘 $-ky$, 감쇠가 막는 힘 $-b\dot{y}$
2. 뉴턴 제2법칙을 적음 — $\sum F = m\ddot{y}$
3. 부호를 붙여 대입 — $f - ky - b\dot{y} = m\ddot{y}$
4. 미분 항을 왼쪽으로 모음 — $m\ddot{y} + b\dot{y} + ky = f$
5. 라플라스 변환을 취함 (초기값 0). 미분 한 번이 s 한 개임

시간 영역	라플라스 영역
$y(t)$	$Y(s)$
$\dot{y}(t)$	$sY(s)$
$\ddot{y}(t)$	$s^2Y(s)$

- 항별로 바꿔 적음 — $ms^2Y + bsY + kY = F$
- Y 로 묶음 — $(ms^2 + bs + k)Y = F$
- 양변을 F 와 $(ms^2 + bs + k)$ 로 나눔 — $\frac{Y}{F} = \frac{1}{ms^2 + bs + k}$

전달함수가 무엇인지 한 줄로

- 입력을 넣으면 출력이 이렇게 나온다는 약속을 s 의 분수 하나로 적은 것
- 자판기와 같음 — 안에서 무슨 일이 일어나는지는 덮어 두고, 넣은 것과 나온 것의 관계만 적음
- 새 물리가 아님. 4번 식과 8번 식은 같은 것을 두 가지로 적은 것임 (실습 B 절에서 두 줄이 겹치는 것으로 확인)

정상상태 확인 — 0.5 m 가 나오는가

- 멈춘 뒤에는 위치가 변하지 않으므로 $\dot{y} = \ddot{y} = 0$
- 남는 것은 $ky = f$
- $y_{ss} = f/k = 1/2 = 0.5$ m — 현상과 맞음
- 실측 (`w04_pid_compare('plant')`), $f = 1$ N 계단, 10 s): **0.50003 m**

1-4. 두 개의 다이얼 — ζ 와 ω_n

현상 — 다이얼 하나로 출렁임이 사라진다

- `zeta` 를 0.2 로 두고 Run 하면 크게 출렁이고, 1 로 두면 한 번도 넘지 않음
- 실측 (`w04_pid_compare('zeta')`), 단위 계단, $\omega_n = 1.4142$ 고정, 15 s)

ζ	오버슈트 [%]	상승 10~90 % [s]	정착 2 % [s]
0.2	55.5	0.84	12.58
0.5	16.3	1.16	5.71
0.7071	4.32	1.52	4.22
1.0	0	2.37	4.13

- ζ 는 속업소버의 세기, ω_n 은 엔진의 힘에 해당함

결과 식

$$\frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \implies \omega_n = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{b}{2\sqrt{mk}}$$

유도 — 한 줄씩

- 2차계의 표준형을 적음 — $\frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$. 분모의 계수 두 개가 곧 $\zeta\omega_n$ 임
- 1-3 의 플랜트 분모를 표준형과 같은 꼴로 만들기 위해 양변을 m 으로 나눔 — $s^2 + \frac{b}{m}s + \frac{k}{m}$
- 두 식의 계수를 하나씩 맞대어 놓음

자리	표준형	플랜트
s^0	ω_n^2	k/m
s^1	$2\zeta\omega_n$	b/m

4. 위 줄에서 $\omega_n = \sqrt{k/m}$

5. 아래 줄에 그 ω_n 을 넣고 ζ 로 뭉 — $\zeta = \frac{b}{2m\omega_n} = \frac{b}{2\sqrt{mk}}$

6. 값을 대입 — $\omega_n = \sqrt{2/1} = 1.4142 \text{ rad/s}$, $\zeta = 2/(2\sqrt{2}) = 0.7071$

오버슈트는 ζ 만으로 정해진다 (결과식)

$$M_p = 100 e^{-\pi\zeta/\sqrt{1-\zeta^2}} [\%]$$

- 유도는 참조 강의 1(제어공학)에 있음. 본 절에서는 **결과를 표로 확인함**

ζ	손계산 M_p [%]	측정 [%]
0.2	52.7	55.5
0.5	16.3	16.3
0.7071	4.32	4.32
1.0	0	0

i $\zeta = 0.2$ 만 3 %p 어긋나는 이유

15 s 실행 구간 안에 완전히 가라앉지 않아 측정 구간이 짧음. 정착시간도 12.58 s 대 손계산 14.14 s 로 어긋남
StopTime 을 40 으로 늘리면 손계산에 붙음

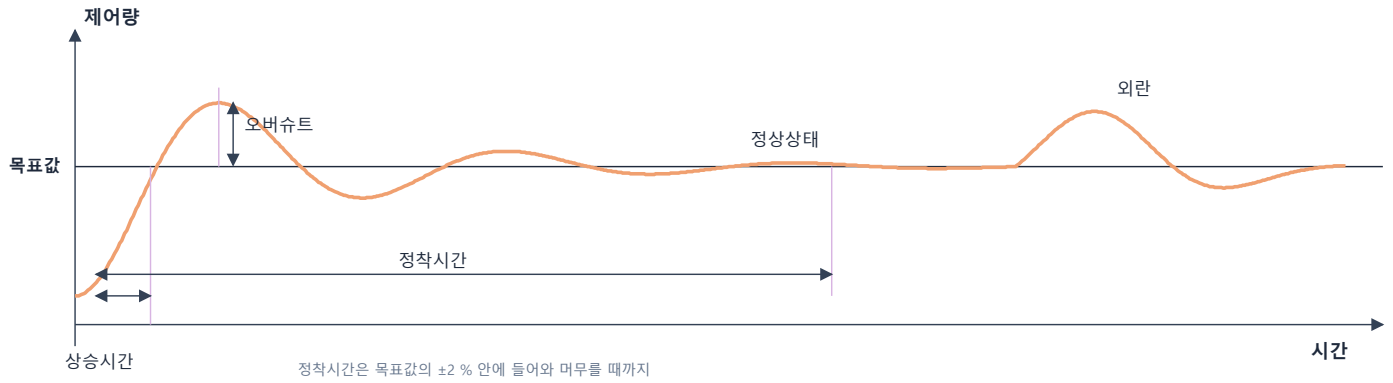
ω_n 은 빠르기만 정한다

- 실측 (`w04_pid_compare('wn')`), $\zeta = 0.7071$ 고정)

ω_n [rad/s]	오버슈트 [%]	상승 [s]	정착 2 % [s]
0.7071	4.46	3.03	8.53
1.4142	4.32	1.52	4.22
2.8284	4.32	0.76	2.11

- ω_n 을 두 배로 하면 상승·정착이 **정확히 절반**이 되고 오버슈트는 4.32 % 로 그대로임
- ζ 는 모양을, ω_n 은 빠르기를 정함. 계인을 설계한다는 것은 이 두 숫자를 고르는 일임 (1-15 극배치)

1-5. 응답을 판정하는 다섯 숫자



용어	뜻
상승시간 (rise time)	목표의 10 % 에서 90 % 까지 가는 데 걸린 시간
첨두시간 (peak time)	최고값에 달는 시각
오버슈트 (overshoot)	최종값을 얼마나 넘어섰는가 [%]
정착시간 (settling time)	최종값의 $\pm 2\%$ 안에 들어와 머무를 때까지
정상상태 오차	충분히 기다린 뒤에도 남는 목표와의 차이

정착시간 어림식 — 왜 $4/(\zeta\omega_n)$ 인가

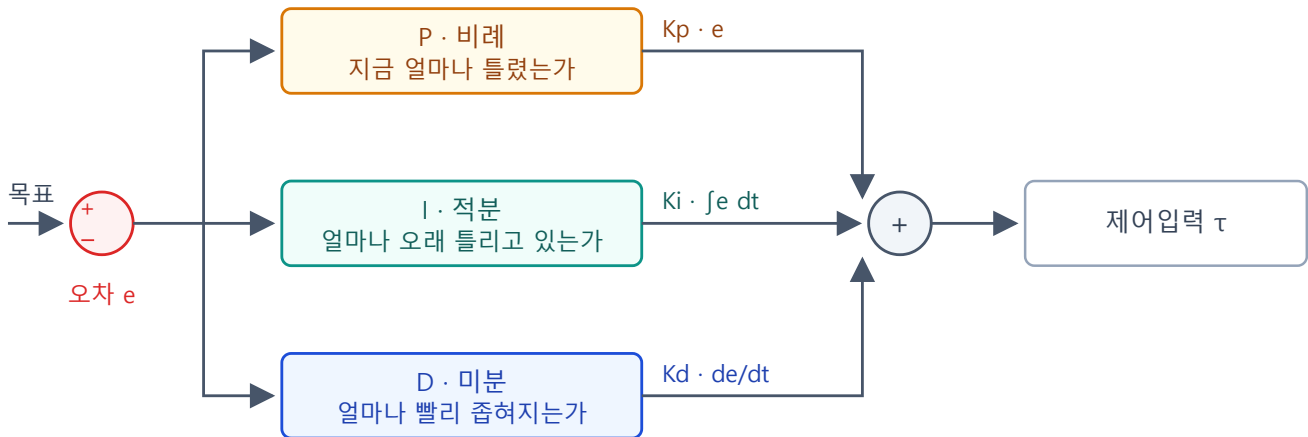
1. 2차계의 진동은 포락선 $e^{-\zeta\omega_n t}$ 안에서 줄어듦
 2. 포락선이 2 % 로 줄어드는 시각을 t_s 라 하면 $e^{-\zeta\omega_n t_s} = 0.02$
 3. 양변에 자연로그를 취함 — $-\zeta\omega_n t_s = \ln 0.02 = -3.91$
 4. $t_s = 3.91/(\zeta\omega_n) \approx \frac{4}{\zeta\omega_n}$
- 계산: $\zeta = 0.7071, \omega_n = 1.4142 \rightarrow t_s \approx 4/1.0 = 4.00$ s. 측정 4.22 s

! 외란은 스텝응답이 아니라 끝난 뒤에 옴

그림 오른쪽에서 곡선이 한 번 솟았다 가라앉는 것이 외란임. 목표는 그대로인데 파도·조류가 배를 밀어낸 것 되먹임이 그것을 스스로 메우는지가 제어기의 진짜 시험

1-6. PID — 오차 하나를 세 가지로 읽는다

PID — 오차 하나를 세 가지로 읽어 더한다



P 는 "지금", I 는 "과거", D 는 "곧 다가올 것"을 본다. 세 시점을 더하는 것이 PID 다.

수식은 이 두 줄이 전부다

$$e(t) = y_d(t) - y(t)$$

$$\tau(t) = \underbrace{K_p e(t)}_{\text{지금}} + \underbrace{K_i \int_0^t e(\sigma) d\sigma}_{\text{과거}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{앞으로}} \Rightarrow C(s) = K_p + \frac{K_i}{s} + K_d s$$

기호	뜻	교과서 표기
y_d	목표값 (desired)	r
y	측정값 · 출력	y
e	오차	e
τ	제어입력 — 힘이나 모멘트	u

⚠ 제어 교과서의 r 과 u 를 본 과목에서는 쓰지 않음

이 과목에서 r 은 **요각속도**, u 는 **전후 속도**임 — 이번 주차의 속도 루프가 $u_{\text{ref}} - u$ 를 쓴 한 페이지에서 u 가 속도이기도 하고 제어입력이기도 하면 읽을 수가 없음
 그래서 목표값은 y_d , 제어입력은 τ 로 씀 (참조 강의 4-5 및 Fossen 과 같은 표기)
 배로 넘어가는 1-13절부터는 제어기 출력을 전후력 X · 요 모멘트 N 으로 적음

오차 하나를 세 가지 방식으로 읽는다

갈래	무엇을 읽는가	시점	비유
P (비례)	지금 얼마나 틀렸는가	현재	스프링 — 멀수록 세게 당김
I (적분)	얼마나 오래 틀리고 있는가	과거	참을성 있는 손 — 맞을 때까지 계속 뭍
D (미분)	얼마나 빨리 좁혀지고 있는가	앞으로	속업소버 — 빨리 다가가면 미리 막음

세 갈래가 각각 못 하는 일

갈래	혼자서는 안 되는 이유	어느 절에서 수치로 보이는가
P	오차가 0 이면 힘도 0. 스프링·항력이 있으면 오차가 남아야만 힘이 유지됨	1-7
I	과거를 쌓으므로 늦게 반응하고 늦게 멈춤 . 혼자 쓰면 크게 지나침	1-9
D	지금의 변화만 보므로 어디에 있는지 모름 . 목표로 데려가지 못함	1-8

- 그래서 셋을 더함. 그것이 PID

1-7. P 가 할 수 있는 것과 없는 것

현상 — 빨라지고, 더 출렁이고, 목표에 못 미친 채 멈춘다

- 실측 (`w04_pid_compare('Kp')`), $K_i = K_d = 0$, 단위 계단 $t = 1\text{ s}, 10\text{ s}$

K_p	정상상태값	오버슈트 [%]	상승 [s]	남는 오차 e_{ss}
2	0.500	16.3	0.82	0.500
10	0.833	38.8	0.38	0.167
50	0.962	64.4	0.16	0.0385

- K_p 를 25 배로 키워도 오차가 0 이 되지 않음. 왜 그런지가 이 절의 요점임

결과 식

$$y_{ss} = \frac{K_p}{k + K_p} y_d, \quad e_{ss} = \frac{k}{k + K_p} y_d, \quad \omega_n = \sqrt{\frac{k + K_p}{m}}, \quad \zeta = \frac{b}{2\sqrt{m(k + K_p)}}$$

유도 — 한 줄씩

- P 제어기는 $\tau = K_p(y_d - y)$ 임. 이것을 1-3 의 운동방정식 $m\ddot{y} + b\dot{y} + ky = \tau$ 에 대입
- $m\ddot{y} + b\dot{y} + ky = K_p(y_d - y)$
- y 가 든 항을 왼쪽으로 모음 — $m\ddot{y} + b\dot{y} + (k + K_p)y = K_p y_d$
- K_p 가 k 옆에 붙었음. 즉 P 제어는 스프링을 하나 더 다는 것과 같음
- 멈춘 뒤를 봄. 정상상태이므로 $\dot{y} = \ddot{y} = 0$
- 남는 것은 $(k + K_p)y_{ss} = K_p y_d$
- 양변을 $(k + K_p)$ 로 나눔 — $y_{ss} = \frac{K_p}{k + K_p} y_d$
- 오차는 $e_{ss} = y_d - y_{ss} = \frac{k}{k + K_p} y_d$ — 스프링이 있는 한 0 이 되지 않음
- 응답 모양은 1-4 의 공식에 $k \rightarrow k + K_p$ 를 넣으면 나옴

검산 — 세 줄 모두 맞는가

K_p	손계산 $y_{ss} = K_p/(k + K_p)$	측정	손계산 ζ	손계산 M_p [%]	측정 [%]
2	$2/4 = 0.500$	0.500	$2/(2\sqrt{4}) = 0.500$	16.3	16.3
10	$10/12 = 0.833$	0.833	$2/(2\sqrt{12}) = 0.289$	38.6	38.8
50	$50/52 = 0.962$	0.962	$2/(2\sqrt{52}) = 0.139$	64.5	64.4

★ P 만으로는 남는 오차를 없앨 수 없음

힘이 오차에 비례하므로, 힘을 계속 내려면 오차가 남아 있어야 함

K_p 를 키우면 오차는 줄지만 0 이 되지 않고, 대신 ζ 가 $1/\sqrt{K_p}$ 로 작아져 오버슈트가 먼저 터짐

1-8. D 는 브레이크

현상 — 출렁임만 깎이고 도착점은 그대로다

- 실측 (`w04_pid_compare('Kd')` , $K_p = 10$, $K_i = 0$)

K_d	정상상태값	오버슈트 [%]	정착 2 % [s]	남는 오차
0	0.833	38.8	3.94	0.167
2	0.833	16.9	1.40	0.167
6	0.833	8.60	1.28	0.167

- 오버슈트와 정착시간이 함께 줄어듦
- 정상상태값은 한 자리도 변하지 않음 (0.833 고정)

결과 식

$$\zeta = \frac{b + K_d}{2\sqrt{m(k + K_p)}}$$

유도 — 한 줄씩

- PD 제어기는 $\tau = K_p e + K_d \dot{e}$ 임
- 목표값이 상수인 동안에는 $\dot{e} = \dot{y}_d - \dot{y} = -\dot{y}$
- 대입하면 $\tau = K_p(y_d - y) - K_d \dot{y}$
- 운동방정식에 넣음 — $m\ddot{y} + b\dot{y} + ky = K_p(y_d - y) - K_d \dot{y}$
- y 와 $\dot{y}$ 가 든 항을 왼쪽으로 모음 — $m\ddot{y} + (b + K_d)\dot{y} + (k + K_p)y = K_p y_d$
- K_d 가 b 옆에 붙었음. 즉 D 제어는 댐퍼를 하나 더 다는 것과 같음
- 1-4 의 공식에 $b \rightarrow b + K_d$ 를 넣으면 위 결과식이 나옴
- 정상상태에서는 $\dot{y} = 0$ 이므로 5번 식의 K_d 항이 사라짐 $\rightarrow y_{ss}$ 는 1-7 과 같음

검산과 남는 의문

K_d	손계산 ζ	손계산 M_p [%]	측정 M_p [%]
0	$2/(2\sqrt{12}) = 0.289$	38.6	38.8
2	$4/(2\sqrt{12}) = 0.577$	10.8	16.9
6	$8/(2\sqrt{12}) = 1.155$	0	8.60

- $K_d = 6$ 은 $\zeta > 1$ 이라 손계산은 오버슈트 0 을 예측하는데 실제로는 8.6 % 가 남음
- 이유: PD 제어기는 분자에도 s 를 넣기 때문임

$$\frac{Y(s)}{Y_d(s)} = \frac{K_d s + K_p}{ms^2 + (b + K_d)s + (k + K_p)} \Rightarrow \text{영점 } s = -\frac{K_p}{K_d}$$

- 분자의 s 가 만드는 것이 영점이며, 영점은 응답의 앞부분을 끌어올려 오버슈트를 더함
- K_d 가 클수록 영점이 원점에 가까워져 그 영향이 커짐 — $K_d = 6$ 이면 $s = -1.67$

- ζ 만으로 오버슈트를 예측하면 PD 에서는 어긋남. 그래서 실측표가 필요함

1-9. I 는 참을성 있는 손

현상 — 오차가 사라지고, 적분항이 스스로 멈춘다

- 실측 (`w04_pid_compare('Ki')`, $K_p = 10$, $K_d = 4$)

K_i	정상상태값	오버슈트 [%]	정착 2 % [s]	남는 오차
0	0.833	9.83	1.32	0.167
4	0.996	0	4.84	0.00433
12	1.000	4.69	1.49	3.4e-06

- $K_i = 4$ 에서 오차가 사라짐. 그때 적분항은 **2 N** 에서 스스로 멈춤
- 그 2 N 은 아무도 알려 주지 않았음. 제어기가 찾아낸 값임

왜 오차가 0 이 되는가 — 세 줄

1. 적분기는 **입력이 0 일 때만** 값이 변하지 않음
 2. 응답이 멈췄다는 것은 적분기도 멈췄다는 뜻이므로 그 지점에서 $e = 0$
 3. 그때 적분기에 쌓여 있는 값이 곧 스프링을 이기는 데 필요한 힘 $ky_d = 2 \times 1 = 2 \text{ N}$
- 즉 I 는 "필요한 힘"을 모른 채로 찾아냄. P 가 못 하던 일임

너무 키우면 불안정해진다 — Routh 판별

1. PID 페루프의 제어입력은 $\tau = K_p e + K_i \int e + K_d \dot{e}$
2. 운동방정식에 넣고 양변을 **한 번 더 미분**하면 적분이 사라져 3차식이 됨
3. 특성방정식은 $ms^3 + (b + K_d)s^2 + (k + K_p)s + K_i = 0$
4. Routh 표를 만들 — 1행에 짝수 차수 계수, 2행에 홀수 차수 계수를 차례로 적음

차수	첫 열	둘째 열
s^3	m	$k + K_p$
s^2	$b + K_d$	K_i
s^1	$\frac{(b + K_d)(k + K_p) - mK_i}{b + K_d}$	0
s^0	K_i	—

5. s^1 행은 위 두 행의 어긋난 곱을 빼서 아래 행의 첫 값으로 나눈 것임 — $\frac{(b + K_d)(k + K_p) - mK_i}{b + K_d}$
6. 안정 조건은 첫 열의 부호가 한 번도 바뀌지 않는 것
7. $K_i > 0$ 이고 $(b + K_d)(k + K_p) - mK_i > 0$ 이어야 함
8. K_i 로 풀면

$$0 < K_i < \frac{(b + K_d)(k + K_p)}{m} = \frac{(2 + 4)(2 + 10)}{1} = 72$$

- 측정한 $K_i = 12$ 는 한계의 6분의 1 이라 안정함
- 실습 C 절의 "해 볼 것"에서 $K_i = 50$ (한계의 0.7 배)과 비교하면 응답이 어떻게 나빠지는지 보임

⚠ I 는 공짜가 아님

- $K_i = 4$: 오차는 없어졌지만 정착시간이 1.32 초에서 4.84 초로 늘어남
- $K_i = 12$: 빨리 메우는 대신 오버슈트가 다시 생김 ($0 \rightarrow 4.69\%$)
- 적분은 과거를 기억하므로 늦게 반응하고 늦게 멈춤